

W3-2팀 설명문서

CO-PASS Engine



CO-PASS Engine은 **Win32 + Direct3D 11 + ImGui** 기반으로 만든 자체 에디터/렌더링 엔진 프로젝트입니다.

현재 프로젝트는 다음 두 가지를 중심으로 구성되어 있습니다.

- **에디터 중심 워크플로우**
 - Outliner, Details, Content Browser, Console, Control Panel 기반 편집
- **실시간 렌더링/자산 파이프라인 실험**
 - Mesh / Sprite / Billboard Text / Outline / Picking / Asset Loading / Scene Serialization

이 저장소는 게임 플레이 프레임워크보다 **에디터 제작, 렌더러 구조화, 씬/에셋 워크플로우 구현**에 더 초점을 둡니다.

1. 프로젝트 개요

CO-PASS Engine은 다음 기능을 직접 구현한 학습형 엔진 프로젝트입니다.

- Win32 기반 에디터 애플리케이션
- Direct3D 11 렌더링 파이프라인
- ImGui 도킹 기반 에디터 UI
- 패널 시스템과 메뉴 레지스트리
- Scene 저장/불러오기 (**.Scene** , JSON 기반)
- AssetManager + Loader 기반 리소스 로딩
- Content Browser / Drag & Drop / Details 연동
- Viewport 카메라, 선택, Gizmo, Outline, AABB 표시
- Sprite / SubUV / Atlas Text 렌더링

2. 주요 특징

에디터 기능

- **Outliner**
 - 현재 씬의 Actor 목록 표시
 - Actor 추가/선택
- **Details**
 - 선택된 Actor/Component의 속성 편집
 - Transform 및 컴포넌트별 수동 property 시스템 지원

- **Content Browser**
 - `Editor/Content` 폴더 인덱싱
 - 폴더 트리, 파일 목록, 검색, 필터, 드래그 앤 드롭
- **Console**
 - 실행 제어, Actor 생성/삭제, 카메라/그리드/뷰모드 변경
- **Control Panel**
 - 카메라 Transform, Projection, FOV
 - View Mode, Show Flags, Grid Spacing, Navigation 속도
- **Shortcuts / About**
 - 단축키 안내와 프로젝트 정보 팝업

렌더링 기능

- Mesh 렌더링
- Sprite 렌더링
- Billboard Text 렌더링
- 선택된 오브젝트 Outline 표시
- Object ID 기반 Picking
- Grid / World Axes / Gizmo / AABB 오버레이
- `Lit / Unlit / Wireframe` View Mode
- Editor Show Flags / Scene Show Flags 토글

데이터/워크플로우 기능

- `.Scene` 저장/불러오기
- `/Game/...` 가상 경로 기반 scene asset path 저장
- AssetManager를 통한 Texture / Font / SpriteAtlas 로딩
- Details에서 AssetPath 직접 입력
- Content Browser에서 Details로 텍스처/폰트/아틀라스 드래그 앤 드롭

3. 기술 스택

- **Language:** C++
- **Platform:** Windows
- **Graphics API:** Direct3D 11
- **Windowing/Input:** Win32 API
- **Editor UI:** Dear ImGui
- **Data:** JSON
- **Project System:** Visual Studio Solution

서드파티 사용 요소:

- Dear ImGui
- nlohmann/json

- DirectXTK Desktop Win10 NuGet package

4. 프로젝트 구조

```

.
├─ Editor/
│  ├─ Content/           # 에디터 실행 시 사용하는 콘텐츠 루트
│  ├─ Resources/        # 아이콘, 로고, rc 리소스
│  ├─ Saved/Config/     # editor.ini
│  └─ Source/
│     ├─ Camera/
│     ├─ Chrome/
│     ├─ Content/
│     ├─ Editor/
│     ├─ Input/
│     ├─ Launch/
│     ├─ Menu/
│     ├─ Panel/
│     ├─ Viewport/
│     └─ ThirdParty/imgui/
├─ Engine/
│  ├─ Resources/Mesh/    # 기본 primitive mesh 데이터
│  └─ Source/
│     ├─ ApplicationCore/
│     ├─ Asset/
│     ├─ Core/
│     ├─ CoreUObject/
│     ├─ Engine/
│     ├─ Renderer/
│     └─ SceneIO/
├─ Docs/
├─ Scripts/
└─ Kraftonjungle_Team2.sln

```

5. 빌드 및 실행

요구 환경

- Windows 10/11
- Visual Studio 2022
- Desktop development with C++
- Direct3D 11 실행 환경

빌드 방법

1. 저장소를 엽니다.
2. `Kraftonjungle_Team2.sln` 을 Visual Studio에서 엽니다.
3. 구성은 보통 `Debug | x64` 를 사용합니다.
4. 솔루션을 빌드합니다.

명령행 빌드:

```
msbuild .\Kraftonjungle_Team2.sln /p:Configuration=Debug /p:Platform=x64 /t:Build /m:1 /nologo /v:minimal
```

프로젝트 파일을 다시 생성해야 하면:

```
.\GenerateProjectFiles.bat
```

실행 루트와 콘텐츠 경로

실행 시 앱 루트는 **Editor** 폴더 기준으로 초기화됩니다.

즉, 기본 콘텐츠 루트는 다음 경로를 사용합니다.

- **Editor/Content**

예를 들어:

- Scene: **Editor/Content/Scenes**
- Font: **Editor/Content/Font**
- Texture: **Editor/Content/Texture**

에디터 설정은 다음 파일에 저장됩니다.

- **Editor/Saved/Config/editor.ini**

6. 에디터 사용 흐름

기본 패널

- **Outliner**: 씬 Actor 목록과 생성
- **Details**: 선택 대상 속성 편집
- **Control Panel**: 카메라/뷰 설정
- **Console**: 명령 기반 조작
- **State**: FPS 등 상태 표시
- **Content Browser**: 에셋 탐색

대표 작업 흐름

1. Outliner에서 Actor를 생성합니다.
2. Viewport 또는 Outliner에서 Actor를 선택합니다.
3. Details에서 Transform / Component 속성을 수정합니다.
4. Content Browser에서 텍스처/폰트/아틀라스를 드래그해 Details에 적용합니다.
5. **.Scene** 으로 저장하고 다시 열어 작업을 이어갑니다.

7. 단축키

실제 단축키 목록은 에디터에서 **Help > Shortcuts** 패널로 확인할 수 있습니다.

현재 코드 기준으로 대표적인 조작은 다음과 같습니다.

- **Mouse Right Drag**: 카메라 회전

- `Mouse Middle Drag` : 카메라 이동
- `Alt + Mouse Left Drag` : 선택 대상을 기준으로 orbit
- `Mouse Wheel` : 카메라 줌 또는 FOV/Ortho 변경
- `W / A / S / D / Q / E` : 카메라 이동
- `F` : 선택 대상에 카메라 포커스
- `Mouse Left Click` : 단일 선택
- `Ctrl + Click` : 선택 토글/다중 선택
- `Delete` : 선택 Actor 삭제
- `Space` : Gizmo 타입 전환

8. Asset System

CO-PASS Engine의 Asset System은 **파일을 직접 읽는 단계**와 **실제 엔진 리소스를 만드는 단계**를 분리합니다.

핵심 파일:

- `Engine/Source/Asset/AssetManager.h`
- `Engine/Source/Asset/AssetLoader.h`
- `Engine/Source/Asset/TextureLoader.cpp`
- `Engine/Source/Asset/FontAtlasLoader.cpp`
- `Engine/Source/Asset/SubUVAtlasLoader.cpp`

개념

- `UAssetManager`
 - Loader 등록
 - Source cache 관리
 - 이미 로드된 Asset cache 관리
- `IAssetLoader`
 - 확장자/타입별 로딩 책임
- `UAsset`
 - 로딩 결과를 담는 엔진 asset 객체
- `Resource`
 - 렌더러가 직접 사용하는 실제 GPU/렌더 자원

지원 Asset 타입

- `Texture`
- `Font`
- `SpriteAtlas`

코드상 enum은 Mesh, Shader, Material도 고려하고 있지만 현재 핵심 구현은 위 세 가지입니다.

로딩 흐름

1. 컴포넌트가 `AssetPath` 를 가짐

2. `ResolveAssetReferences()` 호출
3. `UAssetManager::Load()` 실행
4. Loader가 파일을 읽어 `UAsset` 생성
5. 컴포넌트가 `UAsset` 에서 최종 `Resource*` 를 연결

Source Cache와 Asset Cache

AssetManager는 두 가지 캐시를 분리합니다.

- **Source Cache**
 - 파일 바이트, 파일 크기, 수정 시간, 해시
- **Loaded Asset Cache**
 - 타입 + 경로 + 빌드 시그니처 기반으로 최종 asset 재사용

이 구조 덕분에 같은 파일을 반복 로드할 때 불필요한 디코딩과 재생성을 줄일 수 있습니다.

9. Component와 Asset 연결 방식

현재 프로젝트의 컴포넌트는 보통 다음 구조를 따릅니다.

- 저장용 값: `AssetPath`
- 런타임 값: `Resource*`

예시:

- `USpriteComponent`
 - `TexturePath`
 - `FTextureResource*`
- `UAtlasTextComponent`
 - `FontPath`
 - `FFontResource*`
- `USubUVComponent`
 - `SubUVAtlasPath`
 - `FSubUVAtlasResource*`

즉, 컴포넌트가 `UAsset` 자체를 소유하기보다 경로를 저장하고 필요 시 resolve해서 Resource를 잡는 방식입니다.

10. Content Browser

Content Browser는 `AssetManager` 화면 이 아니라 **에디터 전용 인덱스/브라우저**입니다.

핵심 파일:

- `Editor/Source/Content/EditorContentIndex.h`
- `Editor/Source/Panel/ContentBrowserPanel.cpp`

기능:

- `Editor/Content` 재귀 스캔
- `/Game/...` 가상 경로 생성
- 폴더 트리 / 파일 목록 표시
- 현재 폴더 기준 검색

- 하위 디렉터리 포함 검색
- 타입 필터
- 드래그 앤 드롭

현재 인식하는 항목:

- Scene
- Texture
- Font
- Sprite Atlas
- Unknown File

11. Scene System

씬은 `.Scene` 확장자를 사용하는 **JSON 문서형 파일**입니다.

핵심 파일:

- `Engine/Source/SceneIO/SceneSerializer.h`
- `Engine/Source/SceneIO/SceneSerializer.cpp`
- `Engine/Source/SceneIO/SceneTypeRegistry.cpp`
- `Engine/Source/SceneIO/SceneAssetPath.h`

특징

- Actor / Component 구조 저장
- Component 계층 구조 저장
- 각 Component의 수동 property 시스템 기반 직렬화
- `/Game/...` 기반 asset path 저장
- 로드 시 registry를 통해 실제 Actor/Component 타입 복원
- 알 수 없는 타입은 `UnknownActor`, `UnknownComponent` 로 fallback

Scene 파일의 역할

- 에디터 씬 문서 저장
- AssetManager의 일반 asset cache 항목과는 분리된 문서형 데이터

12. 수동 Property 시스템

이 프로젝트는 Unreal의 `UPROPERTY` 같은 완전한 reflection 시스템 대신, **각 컴포넌트가 직접 어떤 속성을 노출하고 저장할지 선언하는 구조**를 사용합니다.

핵심 파일:

- `Engine/Source/Engine/Component/Core/ComponentProperty.h`

장점:

- Details UI와 Scene 직렬화를 같은 정의로 공유
- 자동 reflection 없이도 확장 가능
- 컴포넌트 작성자가 직접 노출 정책을 제어 가능

예를 들어 텍스트, 텍스처 경로, 색상, 애니메이션 속도 같은 값이 이 시스템을 통해 Details와 `.Scene` 양쪽에 반영됩니다.

13. 렌더링 파이프라인

핵심 파일:

- `Engine/Source/Renderer/RendererModule.cpp`
- `Engine/Source/Renderer/D3D11/D3D11RHI.cpp`
- `Engine/Source/Renderer/D3D11/D3D11MeshBatchRenderer.cpp`
- `Engine/Source/Renderer/D3D11/D3D11SpriteBatchRenderer.cpp`
- `Engine/Source/Renderer/D3D11/D3D11TextBatchRenderer.cpp`
- `Engine/Source/Renderer/D3D11/D3D11OutlineRenderer.cpp`
- `Engine/Source/Renderer/D3D11/D3D11ObjectIdRenderer.cpp`

현재 렌더링은 대략 다음 순서로 진행됩니다.

1. Scene primitive mesh
2. Selection outline
3. Sprite
4. Billboard text
5. Gizmo
6. Grid / Axes / AABB / 기타 line overlay

추가 특징:

- Object ID 오프스크린 렌더링으로 picking 수행
- Selection outline 전용 renderer 존재
- Wireframe 모드에서 선택 오브젝트는 별도 색으로 표시
- UTF-8 텍스트 디코딩을 통해 한글 glyph 렌더링 가능

14. 현재 포함된 Actor / Component 예시

Actor

- `ACubeActor`
- `ASphereActor`
- `AConeActor`
- `ACylinderActor`
- `ARingActor`
- `ATriangleActor`
- `ASpriteActor`
- `AAtlasSpriteActor`
- `AEffectorActor`
- `AFlipbookActor`
- `ATextActor`

- `AUnknownActor`

Component

- Mesh
 - `UCubeComponent`
 - `USphereComponent`
 - `UConeComponent`
 - `UCylinderComponent`
 - `URingComponent`
 - `UTriangleComponent`
 - `UQuadComponent`
- Sprite
 - `USpriteComponent`
 - `UAtlasComponent`
 - `USubUVComponent`
 - `USubUVAnimatedComponent`
- Text
 - `UAtlasTextComponent`
 - `UUUIDComponent`
- Core
 - `USceneComponent`
 - `UPrimitiveComponent`
 - `UUnknownComponent`

15. Console 명령

Console 패널에는 에디터 상태를 바꾸는 명령들이 연결되어 있습니다.

예시:

```
scene.new
scene.save
scene.open "Editor/Content/Scenes/Sample.Scene"
actor.spawn cube 3
actor.spawn sphere 2
actor.delete_selected
select.clear
select.focus
camera.reset
camera.speed 300
camera.rot_speed 0.2
grid.spacing 50
viewmode wireframe
show.grid on
show.outline off
```

```
content.refresh
content.find font
```

16. 알려진 한계 / 현재 상태

현재 프로젝트는 학습형 엔진/에디터이기 때문에 다음 성격을 가집니다.

- Windows + D3D11 환경에 맞춰져 있음
- 완전한 reflection 시스템은 없음
- Undo/Redo 시스템 없음
- 멀티스레드 asset pipeline 없음
- 게임 런타임보다 에디터/렌더링 구조 실험에 더 집중

즉, 범용 상용 엔진보다 **직접 설계하고 구조를 이해하는 목적**에 적합합니다.

17. 문서

추가 문서는 `Docs/` 폴더에 정리합니다.

18. 라이선스

이 프로젝트의 코드는 **MIT License**를 따릅니다.

자세한 내용은 [LICENSE](#)를 참고하세요.

서드파티 라이브러리는 각 라이브러리의 라이선스 조건을 따릅니다.

CO-PASS Engine

Level editor and rendering sandbox for the CO-PASS project.